

医疗康复外骨骼机械臂工程研发总方案

更新时间：2026-05-15

本文是面向真实研发的工程方案，不把机械臂当成平台演示项目。它要回答三个问题：

1. 这个机械臂本体应该怎么一步步做出来。
2. 怎么通过 AI 协作平台开发它，同时让平台本身变成可部署、可扩展、可服务所有项目的研发操作系统。
3. 现在是否支持 Linux，以及部署服务器前还必须验收什么。

结论先写清楚：

- 平台和机械臂同等重要：机械臂不是平台的附属 demo，平台也不是机械臂的专用后台。正确关系是：机械臂作为第一个高复杂度真实项目，倒逼平台长出通用的设备调试、数据工厂、Runner 编排、只读桥、强审、证据链和部署能力。
- Linux 路线是正确且当前仓库已有基础的：nanopi-sdk 已经按 Linux can0/SocketCAN 写了配置脚本，NanoPi_ROSNode 是 ROS2 Jazzy/Python 方向，Runner 电脑和训练电脑也天然适合 Linux。
- 但不能说“已经完整生产支持 Linux”：现在还缺平台级 Linux Runner 安装验收、SocketCAN 只读采集验收、ROS 只读桥验收、systemd/日志/权限/环境变量部署清单。
- 第一阶段不要让 AI 或平台直接控制真实硬件：平台先做 CAN 只读调试台、串口只读调试台、ROS 只读桥、数据训练工厂和强审动作门。
- 机械臂研发先从可观测、可回放、可仿真开始，再进入单电机台架、多电机台架、整机空载、假人/治具、最后才是人参与测试。

1. 项目北极星

目标不是做一个能动的玩具机械臂，而是做一个 具身智能医疗康复外骨骼机械臂系统。它最终应能覆盖：

- 上肢康复训练：被动训练、主动助力、阻抗训练、示教复现。
- 日常辅助：桌面取物、递送、抓握杯子/药盒/水瓶等。
- 患者安全：限位、急停、异常阻力识别、痉挛/疲劳风险提示。
- 数据闭环：电机、EMG、IMU、心率、图像、ROS 状态、训练事件都进入数据工厂。
- AI 决策：VLA 是主要高层决策建议者，OpenClaw 是慢速次级辅助和报告/对话层，小模型负责低延迟本地判断；三者都不能绕过人和安全层。

第一版工程目标应收敛为：

平台可看见所有关键数据
-> 数据可采集、可回放、可标注
-> 仿真可跑通任务状态机
-> 单电机安全台架可验证
-> 多电机空载可验证
-> 整机只在强审后逐步开放动作

同时，平台的北极星也必须独立成立：

平台不是机械臂后台
-> 平台是 AI 协作研发操作系统
-> 任意项目都能建工作区、接 Runner、接仓库、接设备/数据/证据
-> 机械臂只是第一个把这些能力打穿的重型项目

因此后续所有页面、接口和数据模型都要过一道问题：如果明天项目不是机械臂，而是无人机、汽车 ECU、工业机器人、IoT 网关、医疗检测设备、游戏引擎或 SaaS 产品，这个能力还能不能复用。不能复用的，只能放进机械臂项目模板，不能塞进平台核心。

2. 已有资料的有效结论

2.1 两份文档

旧《开发计划文档》给出了早期 PSoC Edge E84、APP、OpenClaw、CAN、传感器节点、移动支架和 5 轴机械臂设想。它的价值是硬件愿景完整，但有些地方太早假设 OpenClaw/APP 可以直接控制。

《总设计文档 v2.0》更合理，已经把系统改成：

- PSoC Edge E84：快、稳、本地安全、小模型、CAN。
- NanoPi M5：Linux、ROS2、相机、状态机、MoveIt、日志。
- 训练/Runner 电脑：VLA、训练、仿真、大模型、数据处理。
- APP/OpenClaw：交互、慢速辅助编排、报告。

这份文档的核心判断需要调整为：英飞凌负责快和稳，NanoPi 负责管和算，VLA 负责主要高层决策建议，OpenClaw 只做慢速次级辅助、对话和报告。OpenClaw 响应慢且安全性不足，不能放在实时链路、不能作为主决策者，也不能直接调用硬件动作。

2.2 两张图

手绘图说明你想要的是一个多智能节点系统：

- VLA/电脑、APP、OpenClaw、英飞凌、NanoPi、语音、摄像头、小模型都参与。
- 原图里 VLA 端输入图像、语音指令、电机数据并输出 action；现在要把这里改成“VLA 输出高层阶段建议，真实执行必须经过仿真/安全层/人工确认”。
- 英飞凌与 NanoPi/CAN/语音/播报/小模型之间有大量数据交换。

微信计划截图说明平台化步骤很明确：

- 先读仓库各分支。
- 创建 NPC/知识库/技能。
- 加 CAN 总线调试台，显示主从 ID 和数据。
- 加串口调试助手。
- 通过 CAN 按采样频率采集电机和传感器数据，送数据工厂训练集。
- 调训练电脑做模型训练。
- 打通 ROS 只读桥，先做辅助仿真。
- 对接 VLA 与 ROS。
- 后续加入 4010 小电机/FOC 调试入口。
- 硬件接线和 VLA/仿真同步推进。

现在用户要求不再调用 NPC，所以这些分工先由 Codex 主线程写成工作包，等切换新项目工作区后再落到平台。

3. GitHub 仓库分支判断

仓库：<https://github.com/ChillAmnesiac/Medical-Rehabilitation-Manipulator>

分支	当前价值	工程判断
APP	Android/Kotlin/Compose，蓝牙/HTTP/OpenClaw、协议解析	可保留为治疗师/患者交互端，但真实运动控制要降级成申请/确认/显示，不应直连危险命令

分支	当前价值	工程判断
C8T6	STM32F103 传感器节点，CAN/USART/ADC/IMU/心率方向	适合作为 EMG/IMU/心率采集节点，需要重写 CAN 帧协议和采样时间戳
M33	PSoC M33 实时控制，CAN、安全、传感器、控制层	是实时安全核心。现有旧 CAN ID 和新控制层要统一到 5 电机 + 传感器 + 支架
M55	模型管理、语音服务、TFLM 槽位	适合跑 EMG 意图、异常检测、动作阶段、唤醒词等小模型
NanoPi_ROSNode	ROS2 Jazzy、相机、HTTP bridge	Linux/ROS 基础成立，但当前桥有写命令入口，第一阶段必须改成只读或强审
ROS_VLA_WebSocket	Node WebSocket 桥，ROS/VLA/Infineon 角色	可作为平台观测/中继参考，VLA 输出不能直接变成硬件动作
ai	VLA task resolver，resolved task schema	适合做高层任务理解和阶段建议，不适合直接输出关节控制
nanopi-sdk	NanoPi MCP2518FD/SocketCAN 配置	说明 Linux can0 路线已开始，但要补真实 NanoPi 安装验收
wake-word-model	唤醒词训练/部署/报告	可作为小模型流水线模板，不是第一阶段主线
PCB	电源板和 STM32 板资料	需要 BOM、接口、供电、保护设计后才能进入硬件执行

关键代码证据：

- nanopi-sdk 的 setup-can.sh 使用 ip link set can0 type can bitrate \${BITRATE} 和 candump can0，这就是 Linux SocketCAN 路线。
- NanoPi_ROSNode 的 HTTP bridge 有 /status 只读端点，但也有 /mode、/control、/api/command 等写入口，部署前必须上强审。
- M33 旧头文件已有 0x100/0x101/0x102 电机和 0x200 传感器 ID，但需要升级到 5 轴、支架、电源、安全节点。
- M33 新控制层已经有 control_motor_enable、stop、set_zero、position_control、read/write_parameter 等危险能力，平台必须把它们分成只读、草案、强审执行。
- M55 有 WAKE_WORD/VAD/ASR_FRONTEND/IMU/EMG/FUSION 六个模型槽，适合形成平台模型发布候选链。
- ai 分支的 ResolvedTask 已经是 phase/confidence/need_confirmation/question 这种高层结构，方向正确。

4. 最终系统架构

推荐采用四层控制、三层智能、一条数据闭环。

4.1 四层控制

- 第 4 层：平台 / APP / OpenClaw / VLA
- 任务理解、协作、报告、数据工厂、训练、强审
- 第 3 层：Runner 电脑 / NanoPi Linux / ROS2
- ROS、状态机、MoveIt、相机、SocketCAN、串口、rosviz、只读桥
- 第 2 层：PSoC Edge E84 M33/M55 / C8T6
- 实时控制、小模型、安全保护、传感器采样、CAN 协议
- 第 1 层：电机驱动器 / 传感器 / 支架 / 电源 / 急停
- 实际执行、反馈、限位、硬件保护

4.2 三层智能

智能层	放在哪里	应做什么	不应做什么
低延迟小模型	PSoC M55/M33	EMG 意图、异常检测、动作阶段、唤醒词	VLA、复杂视觉、直接医疗结论
机器人系统智能	NanoPi/ROS	状态机、运动学、规划、安全监督、数据记录	高频电流闭环、绕过 PSoC 安全

智能层	放在哪里	应做什么	不应做什么
高层语义智能	Runner 电脑/VLA/OpenClaw	VLA 做主要任务决策建议；OpenClaw 做慢速解释、报告、次级建议	直接发 CAN/ROS/串口写命令，OpenClaw 进入实时链路

4.3 数据闭环

CAN / 串口 / ROS / 图像 / 语音 / APP 操作

- > 平台采样任务
- > 数据工厂批次
- > AI 预标注
- > 人工确认
- > 训练任务
- > 模型候选
- > 强审发布
- > 回到 PSoC/NanoPi/APP

这条闭环是项目能不能长期进化的关键。机械臂不是一次写完控制代码，而是靠数据不断迭代患者意图识别、异常识别、动作阶段识别和抓取策略。

5. Linux 支持判断

5.1 现在能确定支持的部分

可以明确支持 Linux 的方向：

- 平台服务器可部署 Linux。
- Runner 电脑可用 Linux。
- NanoPi M5 必然走 Linux。
- ROS2 Jazzy 是 Linux 主线。
- SocketCAN 是 Linux 主线，can0、candump、ip link 已在仓库脚本里出现。
- VLA/训练服务更适合 Linux。

5.2 现在不能假装已经完成的部分

还不能说“完整生产支持 Linux”，因为至少还缺：

- 平台服务端 Linux 部署脚本或 Docker/systemd 验收。
- Linux Runner 安装、注册、开机自启、日志回传。
- NanoPi Runner 能力声明：can0、串口、ROS2、相机、磁盘、网络。
- SocketCAN 只读采集端到端验收。
- ROS 只读桥端到端验收。
- Linux 权限模型：哪些命令允许无 sudo，哪些必须人工审批。
- 服务器部署后的域名、HTTPS、反向代理、持久化数据目录、备份策略。

5.3 部署前 Linux 验收清单

平台服务器：

- Node/Web 构建在 Linux 通过。
- API 服务在 Linux 通过。

- 数据库迁移/初始化通过。
- 文件存储、artifact 预览、日志目录权限正确。
- systemd 或 Docker Compose 能重启恢复。
- 反向代理 HTTPS 正常。

Runner Linux：

- Runner 能注册到平台。
- 能上报 OS、hostname、工作目录、能力。
- 能执行只读命令并回传 stdout/stderr/exit code。
- 能处理长任务和断线重连。
- 危险命令不会绕过平台强审。

NanoPi Linux：

- can0 存在。
- ip -details link show can0 可读。
- candump can0 可只读采样。
- 串口列表可读。
- ROS2 环境可加载。
- ros2 topic list 可读。
- 相机设备可枚举。
- 日志和采样文件能传回平台。

结论：能支持 Linux，但部署前必须做 Linux 真实环境验收。现在最应该补的是平台 Linux Runner/NanoPi Runner 的验收脚本和只读桥。

6. 硬件性能要求

这些指标不是最终医疗产品认证指标，而是第一版工程研发的最低性能门槛。真正用于患者前，还需要更严格的风险分析、临床流程和合规审查。

6.1 机械结构性能

项目	第一版目标	说明
主动自由度	5 轴 + 抓手	肩外展/内收、肩前屈/后伸、大臂旋转、肘屈伸、小臂旋前/旋后
单关节机械限位	必须有	每个主动关节必须有机械硬限位，不能只靠软件限位
运动速度	研发初期极低速	初期以安全观测为主，不追求速度；真实体验前逐级放开
背隙/松动	可测量、可记录	先建立测量方法，再决定是否更换减速器/结构
末端负载	先按轻物体验证	第一阶段只考虑杯子/药盒/水瓶等轻负载，禁止承载人体重量
关节输出能力	待按负载计算确认	不能凭感觉选电机；必须按臂长、负载、减速比、效率、急停余量计算扭矩
减速器背隙	待实测分级	抓取和康复训练对抖动敏感，必须记录每个关节背隙和可接受阈值
断电释放	必须设计	患者不能被机械结构困住
急停	硬件独立	急停不能只走 CAN/软件，必须能切断或抑制真实驱动

项目	第一版目标	说明
支架锁止	必须检测	支架/座椅/升降未锁止时，机械臂不能进入运动模式

6.2 电气与总线性能

项目	第一版目标	说明
CAN 形态	Classical CAN 起步，预留 CAN FD	先看电机/英飞凌/C8T6 是否都支持 CAN FD，再决定升级
CAN 比特率	500 kbps 或 1 Mbps 二选一验证	仓库脚本默认 1 Mbps，旧文档有 500 kbps；必须实测总线长度、节点数、误码
CAN 错误状态	必须采集	平台需要显示 error passive、bus off、rx/tx 错误计数
总线终端	两端 120 欧	布线验收项
节点数量	按实际节点留余量	NanoPi、PSoC、电机、C8T6、支架、电源、调试口都要计入仲裁和带宽
逻辑/动力电源	分域保护	电机电源异常不能拖垮控制板
电源容量	待按峰值电流确认	必须区分连续电流、峰值电流、堵转/限流、急停释放瞬态
电流/温度	每个驱动可读	至少进入只读采集和数据工厂
急停链路	独立于平台	平台只显示急停状态，不作为唯一急停执行者

6.3 计算与通信性能

节点	最低要求	后续要做
PSoC M33	稳定 CAN、故障状态、局部安全降级	完成 5 轴状态帧、故障码、参数只读
PSoC M55	能运行 TFLM 小模型	先跑 EMG/异常/阶段候选，不急着上线
C8T6	稳定 ADC/I2C/CAN/串口日志	加采样序号、时间戳、数据质量字段
NanoPi	Linux、SocketCAN、ROS2、串口、相机枚举	接入平台 Runner，只读采集 can0/ROS/串口
训练电脑	能跑 VLA/训练/仿真任务	负责 VLA 推理、数据训练、仿真回放
平台服务器	Web/API/数据库/文件存储稳定	Linux 部署、备份、健康检查、权限审计

6.4 采样与延迟要求

数据	目标频率	用途
EMG 原始	500-2000 Hz	意图识别、疲劳/异常分析
EMG 特征	50-100 Hz	M55/平台训练输入
IMU	100-200 Hz	姿态、抖动、阶段判断
电机状态	100-500 Hz	角度/速度/电流/温度/故障
ROS 状态	10-100 Hz	joint_states/tf/safety/session
图像	5-30 fps	VLA/视觉定位/抓取确认
心率/血氧	1-10 Hz	训练记录与安全参考，不做实时控制主依据
平台预览	可下采样	UI 展示不应阻塞高频采集

还需要单独建一张“待确认硬件参数表”，包括电机型号、额定/峰值扭矩、额定/峰值电流、额定电压、减速比、背隙、驱动协议、传感器型号、ADC 位数、CAN 收发器、隔离方式、电源容量、急停接线方式、线束长度和总线终端位置。没有这张表，不能进入真实多电机台架。

6.5 硬件后续任务

部分	后续要干什么	验收标准
机械臂本体	关节方案、限位、负载估算、背隙测量、快拆/释放设计	有结构图、限位方案、风险清单
支架/座椅/升降	锁止检测、防夹、底盘稳定、升降刚度	未锁止时运动链路被禁止

部分	后续要干什么	验收标准
电源/急停	电源分域、保险、急停、接地、线束	急停独立有效，平台可读状态
CAN 总线	选定比特率、节点 ID、终端、线束、错误采集	平台只读能看到总线状态和帧流
单电机台架	固定治具、限流、温度、电流、角度状态	先只读，再强审点动
多电机台架	M1-M5 低速空载、联动保护	异常可停，数据可回放

7. 机械本体工程方案

7.1 自由度建议

第一版机械臂建议按 5 主动轴 + 抓手：

1. M1 肩外展/内收。
2. M2 肩前屈/后伸。
3. M3 大臂内旋/外旋。
4. M4 肘屈伸。
5. M5 小臂旋前/旋后。
6. 抓手开合。

这比 3 轴能更接近人体上肢，也比加入完整腕部更现实。腕部自由度先不要上，除非抓取任务证明必须。

7.2 机械设计原则

- 重电机尽量靠近近端，减小末端惯量。
- 肩部双轴必须做刚度和限位，不然整机会抖。
- 每个关节必须有机械硬限位，不只依赖软件限位。
- 需要快拆/离合/断电可释放方案，避免患者被困。
- 抓手要软包覆，避免夹伤。
- 支架底盘要先稳定，再谈助力移动。
- 升降柱要有防夹、上下限位、断电制动。
- 座椅/扶手开合要检测锁止状态，未锁止不能进入动作模式。

7.3 不要过早追求的东西

- 不要第一阶段就做人机共融高速运动。
- 不要第一阶段就上端到端 VLA 控制。
- 不要第一阶段就让 APP 直接发电机目标角。
- 不要第一阶段就让 OpenClaw 直接调用 PSoC 工具移动关节。
- 不要第一阶段就做患者真实康复训练。

8. 电气与 CAN 总线方案

8.1 CAN 拓扑

你提出 NanoPi、英飞凌、Runner 电脑、所有电机、C8T6 都接在 CAN 总线上。工程上建议这样拆：

```
CAN 主干
|-- PSoC Edge E84: 实时控制与安全
|-- NanoPi: SocketCAN 监听/ROS 桥/高层协调
|-- 电机驱动器 M1-M5
|-- 抓手控制器
|-- C8T6 传感器节点
|-- 支架/底盘/电源/急停节点
|-- 调试口: USB-CAN 或 Runner 电脑
```

注意：Runner 电脑不一定长期在 CAN 主干上，它可以通过 USB-CAN 临时监听。长期运行时 NanoPi 和 PSoC 更适合作为板载节点。

8.2 CAN 基本规则

- 先确定是 Classical CAN 还是 CAN FD。第一版如果设备不统一，建议 Classical CAN 8 字节起步。
- 比特率要统一。仓库脚本默认 1 Mbps，旧文档有 500 kbps，第一周必须用示波器/设备手册/总线长度确认。
- 总线两端 120 欧终端电阻。
- 支线尽量短。
- 电机电源和逻辑电源分域。
- 急停不只走 CAN，必须有独立硬件断能或安全输入。
- CAN 写帧必须有平台强审，默认只读。

8.3 推荐 CAN ID

范围	用途
0x080-0x08F	系统心跳、时钟同步、安全广播
0x100-0x10F	M1-M5 电机命令
0x180-0x18F	M1-M5 电机状态反馈
0x200-0x20F	PSoC 聚合状态和控制状态
0x300-0x31F	C8T6 传感器：EMG/IMU/心率/血氧
0x400-0x4FF	支架、底盘、升降柱、座椅锁止、电源
0x500-0x5FF	诊断、版本、参数只读
0x600-0x6FF	参数写入草案/调试，真实写入强审
0x700-0x70F	紧急告警、故障广播

第一版帧协议要能覆盖：

- 设备心跳。
- 状态反馈。
- 故障码。
- 温度/电流/电压。
- 角度/速度/力矩。
- 传感器采样。
- 时间戳或序号。

8.4 平台 CAN 调试台必须显示

- 总线状态：bitrate、error count、bus off、rx/tx rate。
- 节点表：node id、最后心跳、设备类型、固件版本。
- 帧流：CAN ID、DLC、payload、时间戳、方向。

- 解码字段：角度、电流、温度、故障、EMG、IMU。
- 波形：按信号查看。
- 采样任务：频率、持续时间、触发条件、保存位置。
- 强审门：任何 `cansend`/写帧都不能直接执行。

8.5 电气/CAN 后续任务

部分	后续要干什么	验收标准
CAN 协议	建立 CAN ID、字段、单位、频率、方向、风险等级	平台 decoder 能解析 mock 和回放帧
总线物理层	确认线缆、终端、电源、隔离、接地	can0 无 bus off，错误计数可见
节点心跳	PSoC、电机、C8T6、支架、电源都上报心跳	平台设备树能显示在线/离线
参数读写	第一阶段只读参数，写参数走强审	无任何自动参数写入入口
故障码	统一故障码和严重级别	平台能把故障变成观察卡
数据工厂接入	CAN 采样批次保存到 dataset	可按频率、节点、信号回放

9. 串口调试方案

串口不是辅助小功能，它是早期最重要的救命工具。英飞凌、C8T6、电机驱动板、NanoPi 外设都会先通过串口暴露日志。

第一版串口调试台支持：

- 端口枚举。
- 波特率配置。
- 只读日志。
- JSON 行解析。
- 二进制 hex 预览。
- CRC/帧头协议解析插件。
- 错误行标记。
- 采样入数据工厂。
- 发送命令草案。

禁止：

- 未审查直接写串口。
- 未保存日志就让用户手工复制。
- AI 自动猜命令并发送。

9.1 串口后续任务

部分	后续要干什么	验收标准
端口发现	Runner 枚举串口、波特率、设备名、权限	平台能显示可用端口和只读状态
日志解析	支持 JSON 行、ASCII、hex、CRC 帧	错误行可见，不吞日志
协议映射	把 APP/M33/C8T6 的字段映射到统一 stream	数据工厂能识别 EMG/IMU/心率
发送草案	写命令只生成强审卡	页面无直接发送危险命令
回放	保存串口批次并可回放	不接设备也能复现解析

10. ROS 与 NanoPi 方案

10.1 NanoPi 定位

NanoPi 是机器人 Linux 边缘主控：

- ROS2 Jazzy。
- SocketCAN can0。
- 串口采集。
- 相机采集。
- rosbag。
- ROS 只读桥。
- 后续 MoveIt/ros2_control/状态机。

NanoPi 不应负责：

- 高频电流闭环。
- 绕过 PSoC 的真实运动。
- 自动接受 VLA 命令。VLA 只能生成高层阶段建议，必须先进入平台观察卡、仿真/回放、强审和安全状态检查。

10.2 ROS 包建议

```
rehab_robot_description    # URDF/Xacro/mesh
rehab_robot_msgs           # 自定义消息
rehab_can_bridge           # CAN 到 ROS 状态，只读起步
rehab_serial_bridge        # 串口到 ROS/平台，只读起步
rehab_safety_manager       # 安全状态聚合
rehab_therapy_manager      # 康复训练状态机
rehab_moveit_config        # MoveIt 配置
rehab_vla_bridge           # VLA 高层建议
rehab_openclaw_gateway     # OpenClaw 白名单
rehab_data_logger          # rosbag/平台数据上报
```

10.3 ROS 只读桥第一版

允许：

- `ros2 topic list`
- `ros2 node list`
- `ros2 topic echo --once /joint_states`
- `ros2 param dump` 只读
- rosbag 文件索引
- 抽样图片元信息

禁止自动：

- `ros2 topic pub`
- `ros2 service call`
- `ros2 action send_goal`
- 改参数

- launch/restart 节点

流程：

- ROS 只读桥读到状态
- > 平台生成观察卡
- > 发送给 ROS Runner 任务
- > Runner 分析/仿真/生成建议
- > 涉及写操作则进入强审

10.4 ROS/NanoPi 后续任务

部分	后续要干什么	验收标准
NanoPi Runner	注册平台、上报 can0/串口/ROS/相机能力	平台能看到 NanoPi 能力和健康状态
ROS 只读桥	topic/node/param/rosbag 只读采样	不发生 publish/service/action
URDF/TF	建 5 轴模型、坐标系、限制范围	RViz/仿真能加载，无穿模基础错误
Movelt/状态机	先仿真搜索、接近、对齐、抓取、回撤	VLA 只影响阶段建议
rosbag	采样、索引、回放、送数据工厂	可从平台定位片段
安全状态	/rehab/safety_state 聚合急停/限位/锁止	任何危险状态阻止运动申请

11. PSoC 与固件方案

11.1 M33

M33 是实时控制和安全核心：

- CAN 驱动。
- 电机状态读取。
- 电机命令执行。
- 故障码聚合。
- 传感器快照。
- 急停/限位处理。
- NanoPi 失联降级。

第一阶段 M33 任务：

- 统一 5 电机 ID。
- 把旧 3 轴接口升级到 5 轴。
- 明确所有危险 API 的安全前置条件。
- 输出只读状态帧。
- 支持参数读取，不急着支持参数写入。
- 所有写入操作保留但在平台侧强审。

11.2 M55

M55 适合跑：

- 唤醒词。
- VAD。
- EMG 意图识别。

- IMU 动作阶段。
- EMG+IMU 融合。
- 异常检测小模型。

第一阶段模型不要追求复杂：

- EMG 先做手工特征 + MLP/TinyTCN。
- 异常先做规则 + 小模型候选。
- 动作阶段先用规则标注，后续训练模型。

11.3 C8T6

C8T6 是传感器节点：

- EMG ADC。
- IMU I2C。
- 心率/血氧 I2C。
- 采样时间戳/序号。
- CAN 打包。
- 串口日志。

第一版不要让 C8T6 决策，只采集、校验、上报。

11.4 PSoC/C8T6 后续任务

部分	后续要干什么	验收标准
M33 状态帧	统一 5 轴、电源、急停、故障、模式状态	平台只读可解析
M33 安全状态机	idle、ready、fault、estop、degraded、manual_review	状态变化有日志和回执
M33 参数读取	先支持只读参数和版本信息	不支持自动写参数
M55 模型槽	明确 wake/VAD/EMG/IMU/fusion 的输入输出	每个模型候选有评估报告
C8T6 采样	EMG/IMU/心率/血氧加序号、时间戳、质量	丢包/异常能被平台发现
固件发布	编译包、版本、变更、风险说明	烧录必须人工强审

12. 数据训练工厂方案

12.1 为什么它重要

机械臂最难的不是让电机动，而是长期安全地理解：

- 患者有没有主动发力。
- 动作有没有完成。
- 当前是不是痉挛/疲劳/卡阻。
- 抓取有没有成功。
- 哪些训练参数对这个患者有效。

这些都依赖高质量数据。数据工厂必须是平台核心工作台。

12.2 数据源

数据源	示例	频率建议
EMG	ch1/ch2/raw/rms/features	500-2000 Hz 原始, 50-100 Hz 特征
IMU	acc/gyro/姿态	100-200 Hz
电机	角度/速度/电流/温度/故障	100-500 Hz
心率血氧	bpm/spo2/质量	1-10 Hz
ROS	joint_states/tf/safety/session	10-100 Hz
图像	顶部/末端相机	5-30 fps
语音	指令文本/唤醒事件	事件流
人工标注	阶段/意图/异常/成功失败	事件流

12.3 第一批数据集

- 1. 空载电机状态数据。
- 2. 单电机台架数据。
- 3. EMG 放松/发力/噪声数据。
- 4. IMU 静止/缓慢移动/抖动数据。
- 5. ROS 仿真 joint_states 数据。
- 6. 抓取模拟图像数据。
- 7. 异常回放数据。

12.4 标签体系

意图标签：

- relax
- shoulder_lift_intent
- elbow_flex_intent
- resist
- fatigue_warning

动作阶段：

- prepare
- assist
- hold
- return
- done

异常：

- normal
- unexpected_resistance
- stuck
- spasm_risk
- need_stop

抓取：

- search
- approach
- align
- grasp
- lift
- handover
- place
- return
- fail

12.5 平台数据工厂 UI

它应该像标注台，不是文档页：

左栏：数据集 / 采样任务 / 标签集 / 数据源
 中间：当前样本，波形，图像帧，事件时间线
 右栏：AI 预标注，低置信片段，QA，训练回执
 底部：采集日志，Runner 回执，导出/训练记录

AI 能做：

- 预标注。
- 找低置信片段。
- 建议清洗。
- 生成训练配置草案。

人必须做：

- 确认标签。
- QA 放行。
- 选择模型。
- 发布训练结果。
- 允许部署到硬件。

13. VLA 主决策、OpenClaw 次级辅助、APP 人工确认

13.1 VLA

VLA 是具身智能层的主要高层决策建议者。它负责理解环境、任务、语音/文本意图、图像和系统状态，然后输出“下一阶段应该做什么”的结构化建议。VLA 不是电机控制器、不是安全控制器、不是 ROS 写入者，也不是最终医疗判断者。

VLA 第一阶段输出：

```
{
  "task_type": "retrieve_object",
  "phase": "approach",
  "object_id": "cup",
  "grasp_type": "side_grasp",
  "speed": "slow",
  "confidence": 0.72,
  "need_confirmation": true,
```

```
"question": "是否抓取桌面右侧杯子? "  
}
```

VLA 可以影响：

- 任务阶段：搜索、接近、对齐、抓取、递交、回撤、暂停、失败恢复。
- 目标对象：杯子、勺子、按钮、训练器具、患者可触达对象。
- 高层动作建议：慢速接近、请求确认、建议重新观察、建议改用人工操作。
- 置信度和确认问题：明确什么地方不确定，以及需要人确认什么。

VLA 不能输出：

- 电机电流。
- CAN payload。
- 直接关节轨迹。
- ROS action goal。
- 固件参数。
- 医疗结论。
- 绕过强审的执行命令。

VLA 的正确位置是“主计划建议层”。它可以触发平台生成观察卡、仿真任务、候选计划、人工确认卡，但不能把建议直接落到真实硬件。

13.2 OpenClaw

OpenClaw 是慢速次级辅助。它响应慢、安全性不足，因此不能被设计成和 VLA 协同时控制，也不能作为主决策者。OpenClaw 更适合做解释、报告、问答、慢速复盘和低风险辅助编排。

OpenClaw 可做：

- 查询状态。
- 生成训练计划。
- 生成报告。
- 请求 VLA 计划。
- 暂停/停止会话草案。
- 把复杂状态翻译成治疗师/研发人员能理解的说明。
- 对低风险任务给出检查清单，例如“这批数据缺少哪些标签”。

OpenClaw 不可直接：

- 发 CAN。
- 改 ROS 参数。
- 启动真实运动。
- 烧录固件。
- 选择最终模型。
- 放行 QA。
- 代替 VLA 做实时阶段判断。

- 代替人批准硬件动作。

平台必须给 OpenClaw 设白名单：只允许读状态、读报告、生成草案、发起待审卡。任何涉及真实设备、部署、模型发布、医疗判断的动作，都只能进入强审队列。

13.3 APP

APP 第一阶段作为：

- 状态显示。
- 训练记录。
- 患者/治疗师交互。
- 人工确认入口。
- 报告查看。

不要作为：

- 直接危险运动遥控器。
- 绕过平台强审的命令入口。

APP 的价值是把复杂系统变成安全、可确认、可追溯的人机界面。它可以让治疗师确认“是否进入下一阶段”，也可以让患者看到状态和报告；但 APP 不能因为在手机上看起来方便，就变成危险动作的快捷入口。

13.4 VLA/OpenClaw/APP 后续任务

模块	下一步要做什么	交付物	验收
VLA task schema	固定任务、阶段、对象、置信度、确认问题、禁止字段	vla_task_suggestion schema	schema 中不存在 CAN payload、关节轨迹、ROS action goal
VLA 确认策略	规定哪些阶段必须人工确认，哪些低置信自动降级	确认规则表	低置信、接近人体、抓取、递交都进入确认
VLA 仿真链路	VLA 建议先进入仿真/回放，不进真实硬件	仿真任务卡	未通过仿真和人审不能进入设备写入
OpenClaw 白名单	限制 OpenClaw 只能读状态、写报告、生成草案	工具白名单	OpenClaw 无 CAN/ROS/串口/firmware 写工具
OpenClaw 报告流	把训练、异常、数据质量、风险整理成报告	报告模板	报告引用证据，不直接放行
APP 确认 UI	做状态、确认、报告、风险提示，不做危险遥控	APP 页面职责表	APP 中危险动作只显示待审，不直接执行
平台强审接入	VLA/OpenClaw/APP 的动作统一进入审批模型	强审事件类型	所有真实动作都有审批人、证据和回执

14. 平台本体方案

平台不能变成机械臂专用系统。平台本体要作为“AI 协作研发操作系统”独立推进，它至少包含八类通用能力：

- 项目工作区：需求、仓库、成员、NPC、Runner、工位、任务和证据统一索引。
- NPC 协作工作台：对话、派工、回执、待审、长文本抽屉、结构化卡片和线程绑定。
- 专业工作台：数据工场、AI 实验室、机器人现场、观测台、能力工坊都是同级工具面，不依附某个具体项目。
- 设备调试工作台：CAN、串口、ROS、日志、对象树、波形、属性抽屉和事件线。
- 数据训练工厂：任意传感器、图像、音频、日志、业务事件、用户行为都能采样、标注、QA、训练和回流。
- 只读桥：ROS、服务器日志、数据库、工控设备、GitHub、CI、云资源、移动端日志都能先只读接入。

- Runner 编排：按机器能力派任务，区分本地电脑、Linux 服务器、NanoPi、训练电脑、移动端构建机、硬件开发机。
- 强审动作门：真实世界写入、部署、发布、付款、医疗结论、硬件动作、生产变更都统一受控。
- 证据链与验收：采样、回放、报告、审批、回执、截图、日志、构建结果都能被项目追溯。

机械臂只是一个项目模板：

- CAN 模板。
- 传感器模板。
- ROS 模板。
- 训练模板。
- 安全模板。

这样以后可以服务无人机、汽车、工业设备、IoT、医疗设备、实验室设备，而不是只服务这一台机械臂。

14.1 平台与机械臂的边界

属于平台核心：

- Runner 注册、能力声明、任务执行、日志回传。
- 工作台布局范式：左栏对象、中间工作区、右栏属性/证据/动作、底部事件。
- 数据集、采样任务、标签、QA、训练回执。
- 只读桥和强审动作门。
- 项目证据链和验收脚本。
- Linux 部署、服务健康检查、备份、权限、审计。

属于机械臂项目模板：

- 具体 CAN ID。
- 具体 ROS topic。
- 具体电机型号和参数。
- EMG/IMU/心率标签体系。
- 康复动作模板。
- PSoC/NanoPi/C8T6 设备配置。
- 医疗安全风险清单。

如果一个功能写死 `rehab_arm`、`PSoC`、`NanoPi`、`CAN 0x100`，它只能在机械臂模板里。如果一个功能表达为 `device`、`bus`、`stream`、`runner`、`dataset`、`approval`、`evidence`，它才配进入平台核心。

14.2 平台部署前必须补的能力

部署服务器前，平台自身还要完成：

1. Linux 部署脚本：Web、API、数据库、文件存储、日志、反向代理。
2. Runner Linux 安装脚本：注册、开机自启、工作目录、日志轮转、能力探测。
3. 项目迁移能力：从 `proj_ai_collab` 切到机械臂项目，不丢成员、NPC、资料、仓库索引和工作台入口。
4. 通用设备工作台：不叫“机械臂控制台”，而叫设备/现场/总线/数据工作台。
5. 通用数据工厂：采样对象可来自硬件，也可来自 Web 产品、APP、后端日志、CI、业务事件。

6. 通用强审：硬件动作只是其中一种，部署生产、模型发布、财务动作、医疗结论也要能复用。
7. 全链路验收脚本：工作台结构、专业页导航、派工回执、数据工厂、Runner、观测台、Linux 部署。

14.3 平台后续任务

平台能力	下一步要做什么	机械臂中的验证方式	通用化要求
设备工作台	做 CAN、串口、ROS、日志对象树和中间工作区	显示机械臂总线和传感器 mock	名称必须是设备/总线/数据，不写死机械臂
数据工厂	采样任务、频率、标签、QA、训练回执闭环	电机/EMG/IMU/图像进入同一数据集	支持任意 stream，不限硬件
Runner 能力声明	Linux、NanoPi、训练电脑、安卓构建机能力上报	NanoPi 只读采集，训练电脑跑 VLA/模型	能力按标签表达，不按机器名硬编码
强审动作门	统一审批硬件、部署、模型、医疗结论	CAN 写帧和 ROS 写入都只生成待审卡	未来可复用到生产部署、付款、云资源
证据链	每个采样、报告、审批、回执可追溯	回放一次抓取训练数据	证据类型通用，不暴露内部路径和原始线程 ID
Linux 部署	Web/API/Runner/日志/备份/健康检查	在服务器完成部署级验收	平台部署不依赖机械臂项目存在

14.4 平台验收标准

验收项	通过标准
不服务单一项目	新建非机械臂项目时，数据工厂、Runner、强审、观测台仍可使用
专业工作台像工具	左栏对象/模式，中间工作内容，右栏属性/证据/动作，底部事件
NPC 工作台不被破坏	对话框、输入框、多 NPC 瓷砖、角色色彩、结构化卡片、待审控件、长文本抽屉仍在
Linux 可部署	Web/API 构建、启动、健康检查、Runner 注册、日志回传通过
强审不可绕过	真实硬件、部署、模型发布、医疗结论均不能被 AI 自动确认

15. 阶段路线

Phase 0：服务器和平台本体验收

目标：

- 平台可 Linux 部署，并且平台能力不是机械臂定制。
- Runner 可 Linux 注册。
- 工作台能展示设备调试台 shell。
- 新项目工作区能承载机械臂，但平台首页/项目页仍适配所有项目。

验收：

- Web/API Linux 构建和启动通过。
- Runner 只读命令回执通过。
- 强审动作门存在。
- 数据工厂、AI 实验室、机器人现场、观测台仍保持通用命名和通用对象模型。

Phase 1：无硬件 mock/replay

目标：

- CAN mock 帧。

- 串口 mock 日志。
- ROS mock topic。
- 数据工厂能建训练集。

验收：

- 不接硬件也能完整跑通采集、展示、保存、标注、训练草案。

Phase 2: NanoPi 只读接入

目标：

- NanoPi Linux Runner 注册。
- can0 只读。
- 串口只读。
- ROS 只读。

验收：

- 只读数据能进平台。
- 没有写 CAN/ROS/串口。

Phase 3: 单电机隔离台架

目标：

- 一台电机。
- 独立电源。
- 机械固定。
- 急停可用。
- 只读状态先通。

验收：

- 电机状态、电流、温度、故障能采集。
- 点动命令只在人工强审后执行。

Phase 4: 多电机空载台架

目标：

- M1-M5 状态采集。
- 机械限位。
- 低速空载动作。

验收：

- 任一异常能停。
- 数据进入训练工厂。

Phase 5: ROS 仿真和数字孪生

目标:

- URDF/Xacro。
- RViz。
- MoveIt。
- 状态机。

验收:

- 仿真轨迹不穿模。
- VLA 只能影响高层阶段。

Phase 6: 整机空载

目标:

- 支架、升降、座椅、抓手、相机接入。
- 无患者空载。

验收:

- 急停、限位、锁止状态真实有效。
- 支架未锁止时机械臂不能动作。

Phase 7: 治具/假人测试

目标:

- 模拟人体约束。
- 低速被动训练。
- 异常阻力测试。

验收:

- 阻力异常能停。
- 训练数据可完整回放。

Phase 8: 受控人工体验

目标:

- 非医疗正式用途。
- 明确知情。
- 专人监护。
- 极低速。

验收:

- 不以此作为医疗有效性证明。

- 只作为工程安全体验。

16. 第一周具体任务

第 1 天：

- 新建机械臂项目工作区。
- 导入两份文档、两张图、仓库分支地图。
- 建安全边界卡。
- 建模块地图。

第 2 天：

- 机器人工作台改为设备调试台 shell。
- 加 CAN/串口/ROS 三个对象树。
- 写操作全部锁住。

第 3 天：

- CAN mock 帧流。
- CAN ID 解码表。
- CAN 采样批次进入数据工厂。

第 4 天：

- 串口 mock 日志。
- JSON 行解析。
- 英飞凌/C8T6 日志 schema。

第 5 天：

- ROS 只读 mock。
- `/joint_states`、`/tf`、`/safety_state` 示例。
- 观察卡 -> Runner 分析任务。

第 6 天：

- 数据工厂样本台。
- 波形、事件、标签、AI 预标注。

第 7 天：

- Linux 部署验收脚本草案。
- NanoPi Runner 验收清单。
- 第一轮平台全链路回归。

17. NPC 分工、知识库与 Skill 方案

这一节不是要求现在立刻调用 NPC，而是给未来切到机械臂项目工作区后使用。平台要支持“一个项目一套公司地图”，机械臂只是其中一个项目模板；同样的 NPC/知识库/skill 结构以后可以替换成别的行业角色。

17.1 推荐 NPC 组织表

NPC	工位	核心职责	第一阶段交付物	不允许自动做
Boss / 系统总工 NPC	项目指挥	拆目标、排优先级、识别依赖、收口验收、保护平台通用性	项目路线图、每周验收表、风险升级清单	替人批准硬件动作、医疗结论、部署上线
平台产品架构 NPC	平台本体	保证平台不是机械臂专用系统，抽象工作台、Runner、数据、强审对象	平台对象模型、页面边界、通用能力清单	把机械臂字段写死进平台核心
仓库/配置管理 NPC	仓库治理	管理多分支、多模块、文档、代码迁移计划、生成物清理	分支地图、模块索引、迁移计划、CI 草案	直接重构所有分支、删除未知文件
CAN/串口协议 NPC	设备调试	维护 CAN ID、DBC/decoder、串口帧、采样任务	CAN ID 表、串口协议表、只读采集 schema	CAN 写帧、串口写命令、改电机参数
Linux/NanoPi Runner NPC	边缘 Linux	NanoPi、SocketCAN、串口、ROS2 环境、systemd、日志	NanoPi 验收脚本、Runner 能力声明、只读采集脚本	sudo 安装驱动、改 overlay、重启设备、写 ROS
ROS/仿真 NPC	机器人中间层	ROS2 topic、URDF、MoveIt、rosviz、只读桥、仿真回放	ROS 只读桥、topic 表、仿真任务、rosviz 索引	publish/service/action、真实运动
PSoC M33 固件 NPC	实时控制	M33 CAN、安全、控制层、故障码、降级状态	M33 接口表、安全状态机、只读状态帧	烧录、启用真实控制、参数写入
PSoC M55 小模型 NPC	边缘 AI	M55 模型槽、TFLM、EMG/IMU/异常/唤醒词部署候选	模型输入输出表、量化计划、候选发布包	模型直接刷入硬件并启用
C8T6 传感器 NPC	传感器节点	EMG、IMU、心率/血氧、ADC/I2C/CAN 打包	传感器采样表、CAN 帧、串口日志格式	直接改硬件接线或烧录
机械/电气安全 NPC	硬件安全	关节限位、急停、电源、保险、隔离、布线、支架锁止	安全检查表、上电前清单、台架测试规程	真实上电、运动、采购/下单
数据工厂 NPC	数据闭环	采样频率、数据集、标签、QA、训练回流、manifest	数据 schema、标签集、采样任务、QA 规则	替人确认标注、训练发布
AI/VLA/OpenClaw NPC	高层智能	VLA 主决策建议、OpenClaw 白名单、语音/报告、确认问题	高层任务 schema、确认策略、工具白名单	输出直接电机命令、让 OpenClaw 当主决策者、绕过 ROS/PSoC
Android APP NPC	移动端	APP 状态显示、确认入口、蓝牙/HTTP 协议兼容、报告查看	APP 页面职责、协议适配表、确认 UX	手机端直接危险运动遥控
QA/验收 NPC	测试治理	测试矩阵、回归脚本、验收证据、部署前清单	全链路验收、Linux 验收、强审覆盖表	放行医疗有效性、替人最终验收

建议第一批不要创建太多执行 NPC。实际启动时先用 6 个：

第一批 NPC	覆盖范围
Boss / 系统总工	总控、风险、验收
平台产品架构	平台通用化、工作台、强审
设备调试	CAN、串口、C8T6、PSoC 接口
Linux/ROS Runner	NanoPi、SocketCAN、ROS 只读、仿真
数据与模型	数据工厂、标签、小模型、VLA 数据
QA/安全	医疗/硬件边界、部署验收、强审覆盖

17.2 项目知识库设计

平台应该把知识库分成“平台通用知识库”和“机械臂项目知识库”。这样平台以后换项目时不会被机械臂污染。

平台通用知识库

知识库	内容	给谁用
平台对象模型	Project、NPC、Runner、Task、Artifact、Approval、Receipt、Dataset、Device、Stream	所有 NPC
工作台 UX 契约	NPC 工作台结构、专业工作台布局、左中右下区域规则、不可暴露内部词	平台产品、前端、QA

知识库	内容	给谁用
Runner 与权限	Runner 注册、能力声明、只读/写入边界、执行回 执、断线恢复	Runner、Linux、QA
强审策略	部署、硬件、模型发布、医疗结论、生产变更等审 批类型	Boss、QA、平台架构
数据工厂通用模型	stream、sample、label、manifest、QA、traini ng run、model candidate	数据与模型
部署运维	Linux 部署、systemd/Docker、日志、备份、健康检查	平台、Runner、QA

机械臂项目知识库

知识库	内容	来源
原始需求资料	两份设计文档、两张图、用户补充说明	用户文件
仓库分支地图	APP、C8T6、M33、M55、NanoPi_ROSNode、 ROS_VLA_WebSocket、ai、nanopi-sdk、wake- word-model、PCB	GitHub
CAN/串口协议	CAN ID、帧格式、字段单位、串口日志、decoder	M33/C8T6/nanopi-sdk
ROS/NanoPi	ROS2 topic、node、launch、bridge、rosvag、 SocketCAN	NanoPi_ROSNode
PSoC 固件	M33/M55 分工、控制层、安全层、模型槽	M33/M55
数据与标签	EMG、IMU、电机、心率、图像、语音、动作阶段 、异常标签	数据工厂
安全与测试	上电清单、急停、限位、台架、假人、人工体验边 界	QA/机械/电气
模型与 VLA	EMG 意图、异常检测、动作阶段、VLA resolved task、OpenClaw 工具白名单	ai/wake-word/M55

17.3 每个 NPC 需要加载的知识库

NPC	必读知识库	可选知识库
Boss / 系统总工	平台对象模型、原始需求资料、仓库分支地图、安 全与测试	所有项目知识库摘要
平台产品架构	平台对象模型、工作台 UX 契约、Runner 与权限、强审策略	数据工厂通用模型
仓库/配置管理	仓库分支地图、原始需求资料	平台对象模型、部署运维
CAN/串口协议	CAN/串口协议、M33/C8T6、Runner 与权限、强审策略	数据与标签
Linux/NanoPi Runner	Runner 与权限、部署运维、ROS/NanoPi、CAN/ 串口协议	安全与测试
ROS/仿真	ROS/NanoPi、平台对象模型、强审策略、数据与 标签	模型与 VLA
PSoC M33 固件	PSoC 固件、CAN/串口协议、安全与测试	数据与标签
PSoC M55 小模型	PSoC 固件、模型与 VLA、数据与标签、数据工厂通用模型	安全与测试
C8T6 传感器	CAN/串口协议、数据与标签、安全与测试	PSoC 固件
机械/电气安全	原始需求资料、安全与测试、CAN/串口协议	ROS/NanoPi
数据工厂	数据工厂通用模型、数据与标签、模型与 VLA	CAN/串口协议、ROS/NanoPi
AI/VLA/OpenClaw	模型与 VLA、强审策略、ROS/NanoPi、APP 协议	数据与标签
Android APP	APP 分支资料、强审策略、数据与标签、模型与 VLA	原始需求资料
QA/验收	工作台 UX 契约、强审策略、安全与测试、部署运维	全部摘要

17.4 推荐 skill / 能力包

这里的 skill 是平台内给 NPC 或 Runner 绑定的能力包，不等同于某一个具体工具。第一批推荐如下：

Skill	适用 NPC	作用	权限等级
platform-productizer	Boss、平台架构、QA	保持平台通用化、按项目对象推进、避免 demo 化	只读/文档/低风险代码
repo-branch-scout	仓库/配置管理	只读扫描 GitHub 多分支、生成模块地图和风险表	只读
device-bus-can-readonly	CAN/串口、Linux Runner	SocketCAN/candump、CAN 帧解析、采样批次	只读
serial-log-readonly	CAN/串口、C8T6、PSoC	串口日志采集、JSON 行/二进制帧解析	只读
ros2-readonly-bridge	ROS、Linux Runner、QA	topic/node/param/rosbag 只读采样和观察卡	只读
runner-linux-ops	Linux Runner、QA	Linux Runner 安装检查、systemd、日志、能力探测	只读默认，写操作强审
embedded-firmware-review	PSoC、C8T6、QA	固件接口审查、状态帧、故障码、安全前置条件	只读/代码草案
edge-model-pipeline	M55、数据工厂、AI/VLA	小模型数据、量化、候选包、评估报告	候选生成，发布强审
data-factory-labeling	数据工厂、QA	数据集、标签、低置信、QA、manifest、训练回执	人工确认标签
vla-task-schema	AI/VLA/OpenClaw	高层任务 schema、confirmation、问题生成、白名单工具	只输出建议
safety-gate-review	QA、安全、Boss	硬件/部署/模型/医疗强审卡、风险说明、回滚方案	审查草案
linux-deployment-acceptance	平台、Runner、QA	服务器 Linux 部署验收、健康检查、备份、回滚	部署执行强审
app-human-confirmation-ui	Android APP、平台架构	APP 作为状态/确认/报告入口，不做危险遥控	代码草案

17.5 Skill 与权限边界

平台要把 skill 明确分成四档：

等级	能做什么	例子
L0 只读	读仓库、读日志、读 topic、读 CAN、读串口、读数据集	repo-branch-scout、ros2-readonly-bridge
L1 草案	生成 schema、代码草案、配置草案、训练计划、验收表	platform-productizer、data-factory-labeling
L2 强审写入	需要人批准后才能写文件、改配置、执行部署、写设备	linux-deployment-acceptance、runner-linux-ops
L3 禁止自动	真实上电、真实运动、CAN 写帧、串口写命令、firmware 烧录、医疗放行	只能生成强审卡和操作说明

17.6 项目启动时的 NPC 创建顺序

切换机械臂项目工作区后，不建议一次创建十几个 NPC。推荐顺序：

1. 建 Boss / 系统总工 NPC，加载全部摘要和安全边界。
2. 建 平台产品架构 NPC，先守住平台通用化。
3. 建 设备调试 NPC，推进 CAN/串口/只读采集台。
4. 建 Linux/ROS Runner NPC，推进 NanoPi、ROS 只读桥和 Linux 验收。
5. 建 数据与模型 NPC，推进数据工厂、标签和小模型候选。
6. 建 QA/安全 NPC，覆盖强审、验收、医疗/硬件边界。

等第一阶段闭环通过后，再拆出 PSoC M33、PSoC M55、C8T6、Android APP、机械/电气等专项 NPC。

17.7 NPC/知识库/Skill 后续任务

项目	下一步要做什么	验收
NPC 模板	在平台里做项目级 NPC 组织模板，不绑定机械臂	新项目可从模板复制并替换行业知识库
知识库隔离	平台通用知识库和机械臂知识库分开索引	平台核心不出现机械臂专用字段
Skill 权限	每个 skill 标注 L0/L1/L2/L3 权限	L2/L3 动作不能自动执行
角色启动顺序	机械臂项目先开 6 个核心 NPC，再拆专项	不会一开始产生十几个无人收口的线程
验收回执	每个 NPC 输出必须绑定交付物和证据	没有证据的“已完成”不算完成

18. 风险清单

风险	严重性	对策
AI 直接控制硬件	极高	强审门，VLA 只输出高层建议，OpenClaw 只做慢速次级辅助
CAN 协议混乱	高	先建 CAN ID 表和 decoder，再写固件
Linux 只口头支持	高	服务器/Runner/NanoPi 三套验收脚本
数据散落	高	所有采样进数据工厂
机械结构过重	高	电机近端化、先做负载计算
支架不稳	高	底盘/锁止/升降优先验证
APP 绕过平台	高	APP 作为确认和显示入口
ROS bridge 有写入口	高	第一阶段只读，写入强审
模型过早部署	高	候选、评估、人工发布、二次启用
医疗合规误判	极高	只做工程研发，不宣称医疗效果
NPC 过早拆太细	中	第一阶段只建 6 个核心 NPC，闭环后再拆专项
知识库污染平台核心	高	平台通用知识库和机械臂模板知识库分离
skill 权限过大	高	skill 分 L0/L1/L2/L3，默认只读

19. 当前部署前结论

如果你问“现在确定支持 Linux 吗”，我的严谨回答是：

方向上支持，代码基础也有，但还没完成部署级 Linux 验收。

可以放心把服务器部署目标定为 Linux；NanoPi/ROS/SocketCAN 也应该坚定走 Linux。但在真正服务器部署和机械臂项目切换前，必须补下面四个验收：

1. 平台 Web/API 在 Linux 上构建、启动、健康检查通过。
2. Linux Runner 能注册、执行只读任务、回传日志、断线恢复。
3. NanoPi Runner 能只读采集 `can0`、串口和 ROS topic。
4. 所有 CAN/串口/ROS/firmware/运动写操作都只能生成强审卡，不能自动执行。

只要这四项过了，平台就可以作为机械臂研发的工作底座。之后我们再逐步把机械臂从 mock/replay 推到真实硬件。